形式语言与编译: 第五次作业

班级: 学号:

习题 10.2

答：

本题采用三地址代码翻译。布尔表达式采用短路控制流翻译，即每个条件表达式都带有两个出口标号：真出口和假出口。

(1) 翻译语句

```
while(1<x&&y>1)
    if(-n+2<x*y)a = 100
    else b = a
```

令循环开始标号为 l1，循环体入口为 l3，循环出口为 l7。条件 1<x&&y>1 采用短路计算：若 1<x 为假，则直接跳出循环；若为真，再判断 y>1。

翻译得到的三地址代码为：

```
LABEL l1
IF 1 < x THEN l2 ELSE l7
LABEL l2
IF y > 1 THEN l3 ELSE l7

LABEL l3
t1 = - n
t2 = t1 + 2
t3 = x * y
IF t2 < t3 THEN l4 ELSE l5

LABEL l4
a = 100
GOTO l6

LABEL l5
b = a

LABEL l6
GOTO l1

LABEL l7
```

其中：

$$t1 = -n, \qquad t2 = t1 + 2, \qquad t3 = x * y.$$

所以 -n+2<x*y 被翻译为对 t2<t3 的条件跳转。

该语句的带注释语法树可用缩进形式表示为：

```
S.code
|
|-- while (C) S1
    |
    |-- C.true  = l3
    |-- C.false = l7
    |-- C.code
    |   |
    |   |-- C -> C1 && C2
    |       |
    |       |-- C1.code = IF 1 < x THEN l2 ELSE l7
    |       |-- C2.code = IF y > 1 THEN l3 ELSE l7
    |
    |-- S1.code
        |
        |-- if (C3) S2 else S3
            |
            |-- C3.code
            |   |
            |   |-- t1 = - n
            |   |-- t2 = t1 + 2
            |   |-- t3 = x * y
            |   |-- IF t2 < t3 THEN l4 ELSE l5
            |
            |-- S2.code = a = 100
            |
            |-- S3.code = b = a
```

(2) 翻译语句

```
if(x) while(y) {
    if(x>y)goto next;
    x = x*y
}
else {
    y = x*y;
    next: print y
}
```

这里 else 与最外层的 if(x) 配对。令 l1 为 then 分支入口，l5 为 else 分支入口，l2 为 while 循环判断处，l7 为整个语句出口。源程序中的标号 next 直接作为三地址代码中的标号使用。

翻译得到的三地址代码为：

```
IF x != 0 THEN l1 ELSE l5

LABEL l1
LABEL l2
IF y != 0 THEN l3 ELSE l7

LABEL l3
IF x > y THEN next ELSE l4

LABEL l4
t1 = x * y
x = t1
GOTO l2

LABEL l5
t2 = x * y
y = t2

LABEL next
PRINT y

LABEL l7
```

说明如下：

- 若 x 为真，则进入 while(y)；
- 若 x 为假，则进入 else 分支；
- 在循环体中，若 x>y 为真，则执行 goto next；
- 标号 next 对应的语句为 print y。

该语句的带注释语法树可用缩进形式表示为：

```
S.code
|
|-- if (C) S1 else S2
    |
    |-- C.code = IF x != 0 THEN l1 ELSE l5
    |
    |-- S1.code
```

```
|   |
|   |-- while (C1) S3
|       |
|       |-- C1.code = IF y != 0 THEN l3 ELSE l7
|       |
|       |-- S3.code
|           |
|           |-- block
|               |
|               |-- if (C2) S4
|               |   |
|               |   |-- C2.code = IF x > y THEN next ELSE l4
|               |   |-- S4.code = GOTO next
|               |
|               |-- S5.code
|                   |
|                   |-- t1 = x * y
|                   |-- x = t1
|
|-- S2.code
    |
    |-- block
        |
        |-- t2 = x * y
        |-- y = t2
        |-- LABEL next
        |-- PRINT y
```

习题 10.3

答：

C 语言中 `for` 语句的形式为

$$S \to \text{for}(S; B; S)S.$$

右部从左到右的三个语句分别记为 $S[1], S[2], S[3]$，其中：

$$S[1] \text{ 是初始化语句}, \qquad S[2] \text{ 是步进语句}, \qquad S[3] \text{ 是循环体语句}.$$

条件表达式记为 B。

题目给出的含义为：

```
S[1].code
while(B.code) {
    S[3].code
    S[2].code
}
```

因此，for 语句的三地址代码结构应为：

```
S[1].code
LABEL begin
B.code
LABEL body
S[3].code
S[2].code
GOTO begin
LABEL next
```

其中，begin 是循环条件测试处，body 是循环体入口，next 是循环出口。

(1) 属性文法设计

设每个语句 S 有综合属性 $S.code$，表示该语句生成的三地址代码。设布尔表达式 B 有属性 $B.true$ 和 $B.false$，分别表示真出口和假出口。用 $newl()$ 产生新标号，用 $gen[\cdots]$ 产生一条三地址代码。

对产生式

$$S[0] \to \text{for}(S[1]; B; S[2])S[3]$$

配备如下属性方程：

$$\begin{aligned}
L_1 &= newl(),\\
L_2 &= newl(),\\
L_3 &= newl(),\\
B.true &= L_2,\\
B.false &= L_3,\\
S[0].code &= S[1].code \mathbin{++} gen[\text{LABEL } L_1]\\
&\quad \mathbin{++} B.code\\
&\quad \mathbin{++} gen[\text{LABEL } L_2]\\
&\quad \mathbin{++} S[3].code\\
&\quad \mathbin{++} S[2].code\\
&\quad \mathbin{++} gen[\text{GOTO } L_1]\\
&\quad \mathbin{++} gen[\text{LABEL } L_3].
\end{aligned}$$

其中，$B.code$ 的形式为：

$$B.code = gen[\text{IF } B.place \text{ THEN } B.true \text{ ELSE } B.false],$$

或在关系表达式情形下直接写为：

$$gen[\text{IF } E[1].place \ relop \ E[2].place \text{ THEN } B.true \text{ ELSE } B.false].$$

(2) 翻译语句

考虑语句：

```
for(i = 0; i < 100; i = i + 1) print i
```

令：

$$S[1]: i = 0, \qquad B: i < 100, \qquad S[2]: i = i + 1, \qquad S[3]: print\ i.$$

翻译得到的三地址代码为：

```
i = 0

LABEL l1
IF i < 100 THEN l2 ELSE l3

LABEL l2
PRINT i

t1 = i + 1
i = t1
GOTO l1

LABEL l3
```

其中：

- `l1` 是循环条件测试处；
- `l2` 是循环体入口；
- `l3` 是循环出口；
- `t1` 是计算 `i+1` 的临时变量。

(3) 带注释语法树

该语句的带注释语法树可用缩进形式表示为：

```
S[0].code
|
|-- for ( S[1] ; B ; S[2] ) S[3]
    |
    |-- S[1].code
```

```
    |   |
    |   |-- i = 0
    |
    |-- B.true  = l2
    |-- B.false = l3
    |-- B.code
    |   |
    |   |-- IF i < 100 THEN l2 ELSE l3
    |
    |-- S[2].code
    |   |
    |   |-- t1 = i + 1
    |   |-- i = t1
    |
    |-- S[3].code
        |
        |-- PRINT i
```

按照属性方程合并各部分代码，得到：

```
S[0].code =
    S[1].code
    LABEL l1
    B.code
    LABEL l2
    S[3].code
    S[2].code
    GOTO l1
    LABEL l3
```

即最终三地址代码正是：

```
i = 0
LABEL l1
IF i < 100 THEN l2 ELSE l3
LABEL l2
PRINT i
t1 = i + 1
i = t1
GOTO l1
LABEL l3
```

习题 10.5

答：

题目给出的程序声明如下：

```
int x; float z;
int a[10, 20], b[6];

float bar(int brr[];) {
    float x;
    x = brr[0] + brr[5];
    return x;
}

float foo(int x; float boo(nil); int arr[];) {
    if(x==0) z = sqrt(boo(arr[0],),)
    else return boo(arr[x],);
}
```

为便于计算，假定：

$$sizeof(\text{INT}) = 4, \qquad sizeof(\text{FLO}) = 8,$$

$$sizeof(\text{FUNC}) = 8, \qquad sizeof(\text{ARRPTT}) = 4, \qquad sizeof(\text{FUNPTT}) = 8.$$

其中，ARRPTT 表示数组原型形参，FUNPTT 表示函数原型形参。另外，假定 `sqrt` 是预定义库函数，不登记在本程序的 `@table` 中。

(1) 全局符号表 `@table`

全局声明依次为：

`int x`, `float z`, `int a[10,20]`, `int b[6]`, `bar`, `foo`.

其中：

$$x.offset = 4,$$

$$z.offset = 4 + 8 = 12,$$

$$a.base = 12 + 10 \times 20 \times 4 = 812,$$

$$b.base = 812 + 6 \times 4 = 836.$$

函数登记项各占 8 个存储单位，因此：

$$bar.offset = 844, \qquad foo.offset = 852.$$

所以全局符号表为：

```
@table:(
  outer:NIL; width:852; argc:0; arglist:NIL; rtype:NIL; code:@code;

  entry:(name:x; type:INT; offset:4)
  entry:(name:z; type:FLO; offset:12)

  entry:(name:a; type:ARRAY; base:812; etype:INT;
         dims:2; dim[0]:10; dim[1]:20)

  entry:(name:b; type:ARRAY; base:836; etype:INT;
         dims:1; dim[0]:6)

  entry:(name:bar; type:FUNC; offset:844; mytab:bar@table)
  entry:(name:foo; type:FUNC; offset:852; mytab:foo@table)
)
```

由于全局部分没有可执行语句，所以：

```
@code:[
]
```

(2) 函数 bar 的符号表和三地址代码

函数 bar 的声明为：

```
float bar(int brr[];) {
    float x;
    x = brr[0] + brr[5];
    return x;
}
```

其中，brr 是数组原型形参，类型为 ARRPTT，占用 4 个存储单位；局部变量 x 是 FLO 类型，占用 8 个存储单位。

因此：

$$brr.offset = 4, \qquad x.offset = 12.$$

在 bar@code 中需要使用临时变量：

$$t1, t2, t3, t4, t5, t6.$$

其中，$t1, t2, t3, t4, t5$ 为整型或地址计算临时变量，$t6$ 为浮点临时变量。

```
bar@table:(
  outer:@table; width:40; argc:1; arglist:(brr); rtype:FLO;
  code:bar@code;
```

```
  entry:(name:brr; type:ARRPTT; offset:4; etype:INT)
  entry:(name:x; type:FLO; offset:12)

  entry:(name:t1; type:TEMP; etype:INT; offset:16)
  entry:(name:t2; type:TEMP; etype:INT; offset:20)
  entry:(name:t3; type:TEMP; etype:INT; offset:24)
  entry:(name:t4; type:TEMP; etype:INT; offset:28)
  entry:(name:t5; type:TEMP; etype:INT; offset:32)
  entry:(name:t6; type:TEMP; etype:FLO; offset:40)
)
```

数组形参 `brr[]` 在运行时可视为数组首地址。因此 `brr[0]` 和 `brr[5]` 可分别翻译为地址计算加取数操作。

```
bar@code:[
  t1 = brr + 0;
  t2 = M[t1];

  t3 = brr + 5;
  t4 = M[t3];

  t5 = t2 + t4;
  t6 = CONVERT t5, FLO;

  x = t6;
  RETURN x
]
```

其中：

- `t1` 和 `t3` 分别保存 `brr[0]` 与 `brr[5]` 的地址；
- `t2` 和 `t4` 分别保存两个数组元素的值；
- `t5` 保存整型加法结果；
- `t6` 是将整型结果转换为浮点型后的值。

(3) 函数 `foo` 的符号表和三地址代码

函数 `foo` 的声明为：

```
float foo(int x; float boo(nil); int arr[];) {
    if(x==0) z = sqrt(boo(arr[0],),)
```

```
    else return boo(arr[x],);
}
```

其中：

- x 是整型形参；
- boo 是返回类型为 FLO 的函数原型形参；
- arr 是整型数组原型形参。

故有：

$$x.offset = 4, \qquad boo.offset = 12, \qquad arr.offset = 16.$$

```
foo@table:(
  outer:@table; width:56; argc:3; arglist:(x boo arr); rtype:FLO;
  code:foo@code;

  entry:(name:x; type:INT; offset:4)
  entry:(name:boo; type:FUNPTT; offset:12; rtype:FLO)
  entry:(name:arr; type:ARRPTT; offset:16; etype:INT)

  entry:(name:t1; type:TEMP; etype:INT; offset:20)
  entry:(name:t2; type:TEMP; etype:INT; offset:24)
  entry:(name:t3; type:TEMP; etype:FLO; offset:32)
  entry:(name:t4; type:TEMP; etype:FLO; offset:40)
  entry:(name:t5; type:TEMP; etype:INT; offset:44)
  entry:(name:t6; type:TEMP; etype:INT; offset:48)
  entry:(name:t7; type:TEMP; etype:FLO; offset:56)
)
```

下面翻译函数体。

条件语句：

```
if(x==0) z = sqrt(boo(arr[0],),)
else return boo(arr[x],);
```

令真出口为 l1，假出口为 l2，语句出口为 l3。则三地址代码为：

```
foo@code:[
  IF x == 0 THEN l1 ELSE l2;

  LABEL l1;
  t1 = arr + 0;
  t2 = M[t1];
```

```
  PAR t2;
  t3 = CALL boo, 1;
  PAR t3;
  t4 = CALL sqrt, 1;
  z = t4;
  GOTO l3;

  LABEL l2;
  t5 = arr + x;
  t6 = M[t5];
  PAR t6;
  t7 = CALL boo, 1;
  RETURN t7;

  LABEL l3
]
```

解释如下：

- 在真分支中，先取出 arr[0]，作为实参调用 boo；
- boo(arr[0],) 的返回值作为实参传给 sqrt；
- sqrt 的返回值赋给全局变量 z；
- 在假分支中，先取出 arr[x]，作为实参调用 boo；
- boo(arr[x],) 的返回值作为 foo 的返回值。

(4) 汇总

最终结果为：

```
@table
@code
bar@table
bar@code
foo@table
foo@code
```

其中：

- @table 保存全局变量和全局函数登记项；
- @code 为空，因为全局部分没有执行语句；
- bar@table 保存 bar 的形参、局部变量和临时变量；

- `bar@code` 保存 `bar` 的三地址代码；
- `foo@table` 保存 `foo` 的形参和临时变量；
- `foo@code` 保存 `foo` 的三地址代码。

习题 11.2

答：

题目程序为：

```
int x;

int a[2, 3];    // 假定初始化为 a[i,j]=j-i

int foo(int x;) {
    x = x + a[0,2];
    return x;
}

int g(int r(nil); int y; int b[];) {
    if(y <= x /\ b[3])
        y = g(&r, r(x,), b[],)
    else
        y = r(y,);
    return y;
}

x = 1;
g(&foo, x, a[],)
```

其中，数组初始化为：

$$a[i,j] = j - i.$$

所以：

$$a[0,0] = 0, \quad a[0,1] = 1, \quad a[0,2] = 2,$$

$$a[1,0] = -1, \quad a[1,1] = 0, \quad a[1,2] = 1.$$

按一维展开时：

$$b[3] = a[1,0] = -1.$$

(1) 程序执行过程分析

首先执行全局语句：

$$x = 1.$$

然后调用：

$$g(\&foo, x, a[]).$$

第一次调用记为

$$g_1 = g(\&foo, 1, a[]).$$

在 g_1 中，判断条件为：

$$y \le x \ / \ b[3].$$

此时：

$$y = 1, \qquad x = 1, \qquad b[3] = -1.$$

因此：

$$1 \le 1 \text{ 为真}, \qquad b[3] \neq 0 \text{ 为真}.$$

所以进入 then 分支。

then 分支中需要计算：

$$y = g(\&r, r(x,), b[],).$$

此时 r 实际上指向函数 foo，所以：

$$r(x,) = foo(1).$$

而

$$foo(1) = 1 + a[0,2] = 1 + 2 = 3.$$

因此递归调用为：

$$g_2 = g(\&foo, 3, a[]).$$

在 g_2 中：

$$y = 3, \qquad x = 1.$$

所以：

$$y \le x$$

即

$$3 \le 1$$

为假，于是执行 else 分支：

$$y = r(y,) = foo(3).$$

又因为：

$$foo(3) = 3 + a[0,2] = 3 + 2 = 5.$$

所以 g_2 中最终有：

$$y = 5.$$

因此，整个调用关系为：

$$\varepsilon() \to g_1(\&foo, 1, a[]),$$

并且 g_1 先调用 $foo(1)$，再调用 g_2；g_2 又调用 $foo(3)$。

(2) 活动树

活动树如下：

```
epsilon()
|-- g1(&foo, 1, a[])
| |-- foo(1)
| |-- g2(&foo, 3, a[])
| |-- foo(3)
```

其中：

- $foo(1)$ 用于计算递归调用 g_2 的实参；
- $foo(3)$ 用于执行 g_2 的 else 分支；
- $foo(3)$ 返回值为 5。

(3) 各个活动记录

采用属性表形式表示活动记录。

```
epsilon()@frame:(
  alink:NIL;
  clink:NIL;
  raddr:NIL;

  x:1;

  a[1,2]:1;
  a[1,1]:0;
  a[1,0]:-1;
  a[0,2]:2;
  a[0,1]:1;
  a[0,0]:0;

  foo[1]:epsilon()@frame;
  foo[0]:foo@label;

  g[1]:epsilon()@frame;
  g[0]:g@label
)
```

第一次 g 调用的活动记录为：

```
g1@frame:(
  arg3:&a[0,0];
  arg2:1;
  arg1:&r[0];

  alink:epsilon()@frame;
  clink:epsilon()@frame;
  raddr:ret_to_epsilon;

  r[1]:epsilon()@frame;
  r[0]:foo@label;

  y:1;
  b:&a[0,0];

  t1:3;     // t1 = foo(1)
  t2:_      // t2 = g2(...), 此时尚未返回
)
```

调用 $foo(1)$ 的活动记录为：

```
foo(1)@frame:(
  arg1:1;

  alink:epsilon()@frame;
  clink:g1@frame;
  raddr:ret_to_g1;

  x:3;      // x = 1 + a[0,2] = 3
  t1:2;
  t2:3
)
```

第二次 g 调用的活动记录为：

```
g2@frame:(
  arg3:&a[0,0];
  arg2:3;
  arg1:&r[0];

  alink:epsilon()@frame;
```

```
  clink:g1@frame;
  raddr:ret_to_g1;

  r[1]:epsilon()@frame;
  r[0]:foo@label;

  y:5;      // y = foo(3) = 5
  b:&a[0,0];

  t3:5
)
```

调用 $foo(3)$ 的活动记录为：

```
foo(3)@frame:(
  arg1:3;

  alink:epsilon()@frame;
  clink:g2@frame;
  raddr:ret_to_g2;

  x:5;      // x = 3 + a[0,2] = 5
  t1:2;
  t2:5
)
```

(4) 在 $foo(3)$ 活动结束、但 g_2 活动尚未结束时的栈快照

假定按字编址，栈底单元地址为 500，并且栈从高地址向低地址增长。这里给出一种符合课堂运行时环境约定的栈快照。

此时 $foo(3)$ 已经返回，故 $foo(3)$ 的活动记录已经释放。但是 g_2 仍然活着，所以当前活着的活动为：

$$\varepsilon(), \qquad g_1, \qquad g_2.$$

栈快照如下：

```
500  <访问链>:0                    epsilon()
499  <控制链>:0
498  <返址>:NIL
497  x:1
496  a[1,2]:1
495  a[1,1]:0
494  a[1,0]:-1
493  a[0,2]:2
```

```
492  a[0,1]:1
491  a[0,0]:0
490  foo[1]:epsilon()@frame
489  foo[0]:foo@label
488  g[1]:epsilon()@frame
487  g[0]:g@label

486  <参数3>:491                    g1(...)
485  <参数2>:1
484  <参数1>:479
483  <访问链>:499
482  <控制链>:499
481  <返址>:ret_to_epsilon
480  r[1]:499
479  r[0]:foo@label
478  y:1
477  b:491
476  t1:3
475  t2:_

474  <参数3>:491                    g2(...)
473  <参数2>:3
472  <参数1>:467
471  <访问链>:499
470  <控制链>:482
469  <返址>:ret_to_g1
468  r[1]:499
467  r[0]:foo@label
466  y:5
465  b:491
464  t3:5
```

说明：

- 地址 491 是数组 a 的首地址，即 $a[0,0]$ 的地址；
- g_1 的形参 b 和 g_2 的形参 b 都指向 $a[0,0]$；
- g_2 中的 y 已经由 $foo(3)$ 的返回值更新为 5；
- $foo(3)$ 的活动记录已经释放，所以不再出现在该栈快照中。

习题 11.4

答：

题目程序为：

```
int g(int x; int h(nil); int b[];) {
    int y;
    y = x+1;
    y = b[x];
    y = h(y);
    return y;
}

void f() {
    int r(int y;) {
        return y+2;
    }

    int c[2]; int x; int v;
    c[0] = 11; c[1] = 22; x = 0;
    v = g(x, &r, c[],);
    return v+5;
}

f()
```

下面写出：

$$\{\varepsilon, f, r, g\}@table, \qquad \{\varepsilon, f, r, g\}@code, \qquad \{\varepsilon, f, r, g\}@label.$$

假定一个字为 4 字节，并采用如下约定：

$$sizeof(\text{INT}) = 4, \qquad sizeof(\text{ARRPTT}) = 4,$$

$$sizeof(\text{FUNPTT}) = 8, \qquad sizeof(\text{FUNC}) = 8.$$

其中：

- ARRPTT 表示数组原型形参；
- FUNPTT 表示函数原型形参；
- 函数值由两部分组成：代码地址和访问链环境。

(1) $\varepsilon@table$ **和** $\varepsilon@code$

最外层只有函数 g 和函数 f，因此：

```
epsilon@table:(
  outer:NIL; width:20; argc:0; arglist:NIL; rtype:NIL;
  level:0; code:epsilon@code;

  entry:(name:g; type:FUNC; offset:8;  mytab:g@table)
  entry:(name:f; type:FUNC; offset:16; mytab:f@table)
  entry:(name:t1; type:TEMP; offset:20)
)
```

最外层代码只是调用 f：

```
epsilon@code:[
  t1 = CALL f, 0
]
```

(2) $g@table$ **和** $g@code$

函数 g 的声明为：

```
int g(int x; int h(nil); int b[];) { ... }
```

其中：

- x 是整型形参；
- h 是函数原型形参；
- b 是数组原型形参；
- y 是局部整型变量。

因此：

```
g@table:(
  outer:epsilon@table; width:40; argc:3; arglist:(x h b);
  rtype:INT; level:1; code:g@code;

  entry:(name:x; type:INT; offset:4)
  entry:(name:h; type:FUNPTT; offset:12; rtype:INT)
  entry:(name:b; type:ARRPTT; offset:16; etype:INT)

  entry:(name:y; type:INT; offset:20)

  entry:(name:t1; type:TEMP; offset:24)
  entry:(name:t2; type:TEMP; offset:28)
  entry:(name:t3; type:TEMP; offset:32)
  entry:(name:t4; type:TEMP; offset:36)
```

```
  entry:(name:t5; type:TEMP; offset:40)
)
```

函数体为：

```
y = x+1;
y = b[x];
y = h(y);
return y;
```

翻译为三地址代码：

```
g@code:[
  t1 = x + 1;
  y = t1;

  t2 = b + x;
  t3 = M[t2];
  y = t3;

  PAR y;
  t4 = CALL h, 1;
  y = t4;

  RETURN y
]
```

说明：

- b 是数组原型形参，运行时保存数组首地址；
- $b[x]$ 翻译为先计算地址 $b + x$，再取内存值；
- h 是函数原型形参，调用时使用其代码地址和访问链环境。

(3) $f@table$ **和** $f@code$

函数 f 的声明为：

```
void f() { ... }
```

其中局部函数 r 声明在 f 内部，因此 $r@table.outer = f@table$。

f 中还有：

$$c[2], \quad x, \quad v.$$

因此：

```
f@table:(
  outer:epsilon@table; width:52; argc:0; arglist:NIL;
  rtype:VOID; level:1; code:f@code;

  entry:(name:r; type:FUNC; offset:8; mytab:r@table)

  entry:(name:c; type:ARRAY; base:16; etype:INT;
         dims:1; dim[0]:2)

  entry:(name:x; type:INT; offset:20)
  entry:(name:v; type:INT; offset:24)

  entry:(name:t1; type:TEMP; offset:28)
  entry:(name:t2; type:TEMP; offset:32)
  entry:(name:t3; type:TEMP; offset:36)
  entry:(name:t4; type:TEMP; offset:40)
  entry:(name:t5; type:TEMP; offset:44)
  entry:(name:t6; type:TEMP; offset:48)
  entry:(name:t7; type:TEMP; offset:52)
)
```

f 的函数体为：

```
c[0] = 11; c[1] = 22; x = 0;
v = g(x, &r, c[],);
return v+5;
```

翻译为三地址代码：

```
f@code:[
  t1 = c + 0;
  M[t1] = 11;

  t2 = c + 1;
  M[t2] = 22;

  x = 0;

  PAR c;
  PAR r;
  PAR x;
  t3 = CALL g, 3;
```

```
  v = t3;

  t4 = v + 5;
  RETURN t4
]
```

其中：

- `PAR r` 表示传递函数 r 的函数值，即 $r@label$ 和 f 的当前活动记录；
- `PAR c` 表示传递数组 c 的首地址；
- 由于 r 是在 f 中声明的局部函数，所以传递 r 时必须同时传递它的访问链环境。

(4) $r@table$ **和** $r@code$

函数 r 的声明为：

```
int r(int y;) {
    return y+2;
}
```

因此：

```
r@table:(
  outer:f@table; width:12; argc:1; arglist:(y);
  rtype:INT; level:2; code:r@code;

  entry:(name:y; type:INT; offset:4)
  entry:(name:t1; type:TEMP; offset:8)
  entry:(name:t2; type:TEMP; offset:12)
)
```

翻译为三地址代码：

```
r@code:[
  t1 = y + 2;
  RETURN t1
]
```

(5) $\{\varepsilon, f, r, g\}@label$

下面给出各函数的可执行代码雏形。这里使用符号化的栈帧操作：

- `PROLOGUE name` 表示建立函数 `name` 的活动记录；
- `EPILOGUE name` 表示释放函数 `name` 的活动记录并返回；
- `CALL h,1` 若 h 是函数原型形参，则实际跳转到 $h[0]$，并用 $h[1]$ 构造访问链。

最外层代码为：

```
epsilon@label:
  PROLOGUE epsilon

  t1 = CALL f@label, 0

  EPILOGUE epsilon
  STOP
```

函数 g 的可执行代码为：

```
g@label:
  PROLOGUE g

  t1 = x + 1
  y = t1

  t2 = b + x
  t3 = M[t2]
  y = t3

  PAR y
  t4 = CALL h, 1
  y = t4

  RETURN y

g@exit:
  EPILOGUE g
```

函数 f 的可执行代码为：

```
f@label:
  PROLOGUE f

  t1 = c + 0
  M[t1] = 11

  t2 = c + 1
  M[t2] = 22

  x = 0
```

```
  PAR c
  PAR r
  PAR x
  t3 = CALL g@label, 3
  v = t3

  t4 = v + 5
  RETURN t4

f@exit:
  EPILOGUE f
```

函数 r 的可执行代码为：

```
r@label:
  PROLOGUE r

  t1 = y + 2
  RETURN t1

r@exit:
  EPILOGUE r
```

(6) 运行结果说明

在 f 中：

$$c[0] = 11, \qquad c[1] = 22, \qquad x = 0.$$

调用：

$$g(0, \&r, c[]).$$

在 g 中：

$$y = x + 1 = 1.$$

然后：

$$y = b[x] = b[0] = c[0] = 11.$$

接着：

$$y = h(y) = r(11) = 11 + 2 = 13.$$

所以：

$$g(0, \&r, c[]) = 13.$$

回到 f 后：

$$v = 13.$$

最后：

$$return\ v + 5 = 18.$$

因此，若忽略 `void` 与 `return v+5` 之间的类型不一致，则该程序最终由 f 返回的值为：

$$\boxed{18}.$$